PatchMaster Enterprise — Master Plan (0 → 100)

Codename: *Patching with Proof* **Product:** PatchMaster Enterprise by VYGROUP **Scope of this document:** everything needed to take PatchMaster from where it is today (0) to a complete, proven, market-differentiated product (100). **Schedule policy:** this plan has **no dates and no durations**. Progress is measured in *percent complete and proven*, and every stage ends at a hard acceptance gate.

> **Baseline note.** The "current state" in Section 2 is derived from a code-graph export of the repository (analysis commit `a24e0b83bc41f83f70f691d3ecaca45c4f9dc0fc`, 1,893 nodes, 3,667 edges). It reflects file/function summaries, not running behaviour. Anything marked **[VERIFY]** must be confirmed against the live repo before work starts.

Table of contents

1. Vision, thesis and positioning
2. Current baseline and gap analysis
3. Target architecture
4. How to read the stages (0 → 100 ladder)
5. Stage 0 — Ground truth and hardening (0 → 8)
6. Stage 1 — Proof Loop v1 and Evidence Bundle (8 → 22)
7. Stage 2 — Connectivity: vault, agentless, relay (22 → 34)
8. Stage 3 — Vulnerability intelligence (34 → 48)
9. Stage 4 — Patch engine v3 (48 → 60)
10. Stage 5 — Configuration audit and compliance (60 → 70)
11. Stage 6 — Topology Orchestrator and Offline Reliability Engine (70 → 82)
12. Stage 7 — Trusted Content Chain, scale and resilience (82 → 92)
13. Stage 8 — Productization and release (92 → 100)
14. Cross-cutting tracks
15. Rules for the coding agent (Mimo)
16. Reference scenario (the end-to-end acceptance test)
17. Metrics, risk register and decisions
18. Appendices (schemas, formulas, matrices, sources)

---

1. Vision, thesis and positioning

1.1 One-line vision

> **No patch is promoted without evidence, and every patch ends with a signed, verifiable proof that it was safe, applied, healthy and reversible.**

1.2 The problem

Patch tools answer *"what is missing?"* and *"deploy it."* Operators then carry the real risk: *"will this break production, can I undo it, and can I prove to an auditor what happened?"* Today that gap is filled with spreadsheets, change tickets, hope and screenshots.

Two pressures make it worse:

- Patch volume and exploit speed keep rising, so manual testing and manual evidence cannot keep up.
- Many regulated or isolated estates (banks, government, manufacturing, critical infrastructure, data centres with legacy Unix) cannot use cloud-dependent tooling at all.

1.3 What the market already has (as of October 2026)

Do **not** pitch these as new. Treat them as table stakes or as competitors to out-execute.

Capability	Who already offers it	Implication for PatchMaster
Scan + patch + config assessment in one platform	Qualys VMDR	Parity needed, not a differentiator
AI "patch reliability" scoring	Qualys (Patch Reliability Score), Ivanti (Predictive Remediation)	Prediction alone is not enough; ours must work offline and be backed by proof
Agentless patch and vulnerability management	SecOps	Parity needed
Windows, macOS, Linux, legacy Unix and air-gap in one console	HCL BigFix	Our legacy-Unix strength is real but not unique
Autonomous patching for air-gapped environments	Adaptiva (AirGap for OneSite Patch, July 2026)	Air-gap alone is not unique; our supply-chain verification must be

[VERIFY] Re-check competitor claims before any external positioning. The market moves quickly.

1.4 The thesis (what makes PatchMaster different)

PatchMaster already owns the raw parts of a closed loop: snapshots, canary and ring rollouts, restore drills, drift detection, runbooks, a hash-chained audit log and an Ed25519-capable license engine. No competitor I found combines them into a single, on-prem, offline-capable loop that *refuses to proceed without proof* and *emits a signed record afterwards*. That is the product.

Five differentiators, named so they can be tracked, demoed and defended:

ID	Name	One-line promise
D1	**Proof Loop**	Promotion between rings is gated by evidence, not by a person's hope
D2	**Evidence Bundle**	Every patch job produces a signed, tamper-evident, auditor-ready record
D3	**Offline Reliability Engine**	A patch-risk model that learns from *your* fleet and runs fully on-prem
D4	**Topology Orchestrator**	Patch clusters (PostgreSQL HA, Kubernetes, load-balanced pools) in the right order with health gates
D5	**Trusted Content Chain**	Feeds and patch bundles are signed, verified, provenance-tracked and imported one-way

1.5 Who it is for

- Regulated enterprises that need *evidence*, not just compliance dashboards.
- Isolated or air-gapped estates, including those with AIX, HP-UX, Solaris and FreeBSD.
- Platform and SRE teams that run stateful clusters and cannot afford blind patching.
- MSP-style operators who need multi-tenant isolation.

1.6 Non-goals (explicitly out of scope)

- Becoming a general ITSM or CMDB suite.
- Writing a proprietary vulnerability scanner engine from scratch when open, maintained data and engines exist.
- Cloud-only SaaS dependence. Cloud *connectors* are fine; cloud *dependence* is not.
- Replacing CI/CD tools. The existing CI/CD module stays an internal delivery capability, not a market pillar.

---

2. Current baseline and gap analysis

2.1 What exists today

Layer	Evidence in the export	Status
API	40+ routers under `backend/api/` incl. `hosts_v2`, `jobs_v2`, `bulk_patch`, `register_v2`, `agent_proxy`, `cve`, `compliance`, `sla`, `remediation`, `schedules`, `maintenance`, `ring_rollout`, `canary_testing`, `rolling_restart`, `runbook`, `policies`, `backups`, `restore_drills`, `windows_snapshot`, `mirror_repos`, `packages_router`, `audit`, `reports`, `plugins`, `network_boot`, `provisioning`, `software_kiosk`, `cicd*`	**EXISTS**
Services	`auth.py` (JWT + rotating refresh tokens), `license.py` (HMAC, Ed25519, hardware binding), `multi_tenant.py`, `drift_detector.py`, `prometheus_targets.py`, integrations (Jira, ServiceNow, Slack, Splunk, Sumo Logic, custom webhook)	**EXISTS**
Data	`models/db_models.py` plus dated SQL migrations; tables for ring rollout, restore drills, runbooks, alert tickets, plugins, CI/CD	**EXISTS**
Frontend	React/Vite SPA; `App.jsx` (~4,000 lines) plus ~40 ops pages	**EXISTS** (needs restructuring)
Agent	Flask-based `agent.py` plus `main.py` heartbeat; managers for Windows (PowerShell/winget), apt/yum/zypper/pacman/apk, FreeBSD, AIX, HP-UX, Solaris; PatchRepo server	**EXISTS**
Packaging	Offline wheel bundles, per-platform build scripts, bare-metal installer, prod and HA compose stacks	**EXISTS**
Observability	Prometheus targets, alert rules, Grafana dashboards	**EXISTS**

2.2 Gap analysis (what blocks market credibility)

ID	Gap	Why it matters
G1	No agentless mode (SSH/WinRM)	SecOps's headline feature; blocks "no software on endpoints" buyers
G2	No relay / distribution tier for remote sites	Branch and segmented networks cannot be served efficiently
G3	No host vulnerability *discovery* (CVE module imports feeds and maps, it does not scan)	Without discovery the product depends on external scanners
G4	Prioritization is CVSS-centric	Modern prioritization needs exploit likelihood and active exploitation
G5	No benchmark audit engine (CIS/STIG-style)	Compliance buyers expect pass/fail per control with remediation
G6	macOS not supported beyond OS-family detection	Mixed estates need it
G7	Third-party application patching is thin (Windows via winget only)	Browsers and common apps are the largest real-world exposure
G8	Reboot experience is basic (no user deferral, graceful prompts)	Direct operational pain on endpoints

2.3 Known technical debt (must be cleared before building on top)

[VERIFY] each item against the repo.

- Repository's own issue inventory lists a **SQL injection risk** and other security items; the tracker still shows open priority tiers.
- `frontend/src/App.jsx` is a ~4,000-line root component owning session, license, health and the shell.
- `agent/agent.py` is a monolithic core (API, patching, snapshots, backup, offline install, WSUS, metrics in one file).
- Schema evolves through `create_all` plus hand-dated SQL scripts; there is no single migration framework with up/down guarantees.
- Binary build output and strays are committed (`agent/--skip-agents`, `agent/2.0.0`, `backend/integrations sumo_logic.py` with a space in the name, static agent artifacts).
- Development Dockerfiles are single-stage and use default credentials.

- `backend/version_checker.py` is a disabled stub.
- Security-sensitive features (live command, agent proxy) need an explicit threat model and allowlists.

---

3. Target architecture

3.1 Logical view



3.2 New and extended components

Component	Type	Purpose	Builds on
Credential Vault	NEW	Encrypted storage and scoped use of SSH keys, WinRM and device credentials	Fernet pattern in `cicd_secrets.py`
Connector Layer	NEW	One interface over agent, SSH, WinRM and relay transports	`agent_proxy.py`, `agent.py`
Relay Service	NEW	Jump-host and distribution-server modes for remote networks	`packages_router.py`, `mirror_repos.py`
Content Service	EXTEND	Ingest, verify and serve feeds and patch binaries	`mirror_repos.py`, `cve.py`
Risk Engine	NEW	Score vulnerabilities with severity, exploit likelihood, exploitation, exposure, criticality	`cve.py`, `dashboard.py`
Scan Orchestrator	NEW	Inventory-based matching and unauthenticated port/service/TLS scans	`register_v2.py`, `hosts_v2.py`
Audit Engine	NEW	Benchmark checks, custom profiles, network config audit	`policies.py`, `drift_detector.py`, `compliance.py`
Proof Loop	EXTEND	Gate evaluation between rollout rings	`ring_rollout.py`, `canary_testing.py`
Evidence Ledger	NEW	Signed, hash-chained job evidence bundles	`audit.py` (hash chain), `license.py` (Ed25519)
Reliability Engine	NEW	Offline patch-risk model	`jobs_v2.py` history, `host_timeline.py`
Topology Orchestrator	EXTEND	Service graph, ordered patching, health gates	`rolling_restart.py`, `runbook.py`
Trusted Content Chain	NEW	Signed bundles, verifier, zones	`build_release.py`, `license.py`

3.3 Deployment topologies to support

1. **Single master**, internet-connected.
2. **Single master**, air-gapped (content arrives by signed bundle).
3. **HA master** (load-balanced backends, primary/replica database).
4. **Master + relays** for branch offices and segmented networks.
5. **Multi-tenant master** for MSP operation.
6. **Multiple isolated enclaves**, each with its own master or zone, fed from one online content builder.

3.4 Design principles

1. **Evidence over assertion.** If a claim matters, it has a recorded, verifiable artifact.
2. **Offline first.** Every feature must work with zero internet unless it is explicitly a connector.
3. **Allowlist, never arbitrary.** Remote execution uses vetted command templates, not free-form strings (live command stays admin-only, audited and optional).
4. **Open data, own policy.** Use public vulnerability and benchmark data; own the scoring, gating and proof.
5. **One connector interface.** Features must not care whether a host is reached by agent, SSH, WinRM or relay.
6. **Reversible by default.** No state-changing action ships without a defined undo or an explicit "irreversible" label.
7. **Explainable.** Every score, gate decision and recommendation shows its inputs.

---

4. How to read the stages (0 → 100 ladder)

- **Percent = share of the finished product that is complete *and proven*,** not elapsed effort. A stage is only "done" when its gate passes.
- **Order is risk-driven.** The thesis (Proof Loop and Evidence Bundle) is built **before** parity features, because parity is known work and the thesis is the unknown. *Build order is not release order:* the first public release can wait for parity.
- Each stage has: **Goal → Why here → Work packages → Acceptance gate → Evidence required.**
- Work package IDs are `S{stage}-{n}`. Tags: **[EXISTS]** already there, **[EXTEND]** change existing code, **[NEW]** build from scratch, **[VERIFY]** confirm first.
- No stage opens until the previous gate has passed. Cross-cutting tracks (Section 14) run alongside every stage.
- Every work package must end with its own tests, docs and evidence (Section 15).

Stage	Name	Ladder	Primary outcome
0	Ground truth and hardening	0 → 8	A clean, secure, testable base
1	Proof Loop v1 + Evidence Bundle	8 → 22	The thesis works end to end on today's capabilities
2	Connectivity	22 → 34	Agentless, relay and vault
3	Vulnerability intelligence	34 → 48	Discovery, prioritization, remediation links
4	Patch engine v3	48 → 60	Catalog, third-party apps, macOS, reboot UX
5	Config audit and compliance	60 → 70	Benchmarks, custom profiles, network devices
6	Topology + Reliability	70 → 82	D3 and D4 complete
7	Trusted Content Chain + scale	82 → 92	D5, HA, performance, resilience
8	Productization and release	92 → 100	Shippable, supportable, certifiable

---

5. Stage 0 — Ground truth and hardening (0 → 8)

Goal

A repository you can change with confidence: secure, tested, reproducible and structured.

Why here

Everything later multiplies existing weaknesses. A security hole or a 4,000-line component under new features becomes a liability.

Work packages

S0-1 Security debt closure [EXTEND]

- Reproduce and close every item in the repo's own issue inventory (SQL injection risk, weak-password handling, token handling, hard-coded defaults). **[VERIFY]** status of each.
- Replace any string-built SQL with parameterized queries or ORM constructs; keep `validators.py` but never rely on it as the only defence.
- Add static analysis for Python (bandit-class) and dependency vulnerability scanning in CI.

- Rotate and externalize all default credentials in dev compose files; production stack already demands secrets, keep it that way.
- Add secret scanning to prevent credentials entering the repo.

S0-2 Threat model [NEW]

- Write a STRIDE-style model covering: master API, agent API (token-authenticated Flask), agent proxy, live-command console, license engine, content import, relay, vault.
- Output: `docs/security/threat-model.md` with each threat, mitigation, owner and test.
- Decide and document the agent trust model (per-host tokens, rotation, revocation, mTLS option).

S0-3 Repository hygiene [EXTEND]

- Remove committed build output and strays (`agent/--skip-agents`, `agent/2.0.0`, the space-named integrations duplicate, binary static artifacts). Build artifacts become CI outputs, not source.
- Add `.gitignore`, artifact storage conventions and a reproducible build recipe.
- Normalize file and directory naming; add CODEOWNERS.

S0-4 Migration framework [EXTEND]

- Introduce a single migration tool (Alembic-class) with up/down migrations, replacing ad hoc `create_all` plus dated SQL scripts. Convert the existing dated scripts into the baseline chain.
- Add a migration test: fresh install and upgrade-from-previous-release both end at the same schema.

S0-5 Decompose monoliths [EXTEND]

- Frontend: split `App.jsx` into app shell, routing, auth/session store, license store, notifications, page modules. No behaviour change.
- Agent: split `agent.py` into modules (`api`, `executor`, `managers/*`, `snapshot`, `backup`, `offline`, `metrics`, `auth`). Keep the public HTTP contract identical.
- Backend: split `db_models.py` by domain (hosts, patching, vuln, policy, cicd, mirror, tenancy). Keep imports stable through a package `__init__`.
- Rule: refactors are behaviour-preserving and proven by characterization tests written *first*.

S0-6 Test baseline [EXTEND]

- Run the whole existing pytest and Playwright suites; record pass/fail/flaky; fix flaky tests.

- Add characterization tests around the areas about to be refactored (agent API contract, job lifecycle, heartbeat/register).
- Coverage report as a CI artifact; set a floor that only ratchets upward.

S0-7 CI gates [NEW]

- Pipeline: lint, type-check, unit, API tests, frontend build, smoke E2E, migration test, security scan, package build.
- A change cannot merge unless every gate passes.

S0-8 Environments and fixtures [NEW]

- A reproducible dev stack (compose) plus a **fake fleet**: containerized Debian, Ubuntu, RHEL-family and Alpine hosts running the agent; scripted fixtures for Windows/AIX/Solaris behaviour via recorded command outputs.
- Seed-data generator for hundreds of hosts and thousands of CVEs.

S0-9 Documentation baseline [EXTEND]

- One authoritative architecture doc, one install doc, one developer guide. Remove duplicate status/summary files that contradict each other (many release summaries exist today).

Acceptance gate (Stage 0 → 8%)

1. CI is green on a clean checkout with all gates enabled.
2. Zero open critical/high findings from static analysis and dependency scanning, or each has a documented, owner-approved exception with an expiry condition.
3. Fresh install and upgrade produce an identical schema (migration test passes).
4. `App.jsx` and `agent.py` are decomposed; characterization tests prove behaviour unchanged.
5. No binary artifacts or strays remain in source control.
6. Threat model reviewed and mitigations mapped to tests or tickets.

Evidence required

CI run links, scan reports, migration test output, before/after characterization test results, threat-model document.

---

6. Stage 1 — Proof Loop v1 and Evidence Bundle (8 → 22)

Goal

Demonstrate the thesis end to end using capabilities PatchMaster already has: **gate every ring promotion on evidence and emit a signed record.**

Why here

This is the riskiest and most valuable idea. If it does not work or does not impress, the plan changes early.

Concept: the Proof Loop

```
 Select patch set + target rings
        |
        ▼
 1. PRE-FLIGHT        host reachable • free disk • maintenance window open • no blocking hooks
        |              dependencies resolved • snapshot capability confirmed • rollback path known
        ▼
 2. SNAPSHOT/BACKUP   create restore point (or declare "irreversible" and require override + approval)
        ▼
 3. CANARY RING       apply to canary hosts only
        ▼
 4. HEALTH GATES      service health • error rate • agent heartbeat • custom probes • SLO checks
        ▼
 5. DRIFT CHECK       no unexpected package/service/registry/file drift vs baseline
        ▼
 6. RESTORE PROOF     (policy-driven) restore drill proves the undo path actually works
        ▼
 7. DECISION          PROMOTE next ring | HOLD for approval | ROLL BACK automatically
        ▼
 8. EVIDENCE BUNDLE   signed, hash-chained record of every step above
```

Work packages

S1-1 Gate model [EXTEND]

- Define a `GateDefinition` (type, parameters, severity, timeout, retry, required/advisory) and a `GateResult` (status, observed values, timestamps, raw output pointer).
- Gate types v1: `host_reachable`, `maintenance_window`, `disk_space`, `snapshot_present`, `service_health`, `heartbeat_alive`, `custom_probe` (script/HTTP), `drift_clean`, `restore_drill_passed`, `approval_granted`.

- Extend `ring_rollout_policies.rollout_config` to reference gates per ring and per phase (pre, post).
- Reuse `ring_rollout.py` health and maintenance gates; do not duplicate logic.

S1-2 Decision engine [EXTEND]

- Deterministic evaluator: all *required* gates pass → promote; any required fails → rollback or hold per policy; advisory fails → warn and record.
- Explicit states: `pending`, `running`, `passed`, `failed`, `held`, `rolled_back`, `overridden`.
- Overrides require a role, a reason and a second approver when policy says so; overrides are recorded as such, never hidden.

S1-3 Snapshot and restore abstraction [EXTEND]

- Unify what exists (Windows snapshot, backups, Linux snapshot hooks) behind one `RestorePoint` interface: `create`, `verify`, `restore`, `delete`, `describe`.
- Each platform declares *capabilities* (full snapshot, package-level rollback, config-only, none). The Proof Loop reads capabilities and enforces the "irreversible" rule.

S1-4 Restore drill integration [EXTEND]

- Allow a policy to require a drill on a *sample* of canary hosts before promotion; use `restore_drill_runs` and record measured RTO/RPO versus targets.
- Drill failure is a required-gate failure by default.

S1-5 Evidence Ledger v1 [NEW]

- Table `evidence_bundles` (job id, tenant, created_at, bundle hash, signature, signer key id, previous bundle hash, storage ref) and `evidence_items` (bundle id, step, kind, payload ref, hash).
- Hash chain across bundles (reuse the audit log's SHA-256 chain approach).
- Signing: Ed25519 using a dedicated *evidence signing key*, separate from the license key. Public key shipped with the product for verification.
- Storage: bundle as a single archive (JSON manifest + artifacts), content-addressed.

S1-6 Evidence verifier [NEW]

- Standalone CLI `pm-evidence verify <bundle>` that checks: signature, hash chain, item hashes, completeness against the policy that governed the job.
- Output: PASS/FAIL with reasons. Must run without network and without the master.

S1-7 Evidence viewer UI [NEW]

- Page in the SPA: per-job timeline, gate results, artifacts, signature status, download bundle, "verify" button, export for auditors (PDF summary + raw bundle).

S1-8 Notifications and approvals [EXTEND]

- Checkpoint approvals via existing notification channels and CI/CD approval patterns; approval events become evidence items.

S1-9 Tests [NEW]

- Fake-fleet scenarios: healthy promote, canary health failure → auto rollback, drift detected → hold, restore drill failure → hold, override with second approver, tamper test (modify a bundle → verifier fails).

Acceptance gate (Stage 1 → 22%)

1. A policy with three rings runs against the fake fleet and promotes only after every required gate passes.
2. A forced failure at each gate type produces the correct hold or rollback and is recorded.
3. Every job produces an evidence bundle; `pm-evidence verify` returns PASS on an untouched bundle and FAIL when one byte is changed, a step is removed or the chain is broken.
4. An "irreversible" patch cannot proceed without recorded override and approval.
5. Auditor export renders without network access.

Evidence required

Recorded demo of the three-ring run, test report for all failure injections, sample bundle plus verifier output (PASS and FAIL cases).

---

7. Stage 2 — Connectivity: vault, agentless, relay (22 → 34)

Goal

Reach any asset by the right transport — agent, SSH, WinRM or relay — through one interface, with credentials handled safely.

Why here

Agentless and relay are the largest parity gaps (G1, G2) and every later feature (scanning, audit, patching) rides on this layer.

Work packages

S2-1 Credential Vault [NEW]

- Envelope encryption: per-secret data key encrypted by a master key; master key supplied by environment/file, with a pluggable provider interface for external KMS/HSM later.
- Credential types: SSH password, SSH key (+ passphrase), WinRM (basic/NTLM/Kerberos), SNMP, network-device enable secrets, API tokens.
- Scopes: global, group, site, host; most specific scope wins. Per-use audit entry (who/what/why), never log secrets.
- Rotation support, expiry and "test credential" action (reachability + privilege check).
- Decrypt only inside the connector at execution time; secrets never travel to the browser.

S2-2 Connector interface [NEW]

- Abstract `Connector`: `connect`, `run(command_template, params)`, `put_file`, `get_file`, `facts`, `close`, with timeouts, cancellation and structured results (exit code, stdout, stderr, duration).
- Implementations: `AgentConnector` (wraps current agent proxy), `SshConnector`, `WinRmConnector`, `RelayConnector`.
- **Command templates only.** Operations map to named templates per OS family (e.g. `list_packages`, `apply_updates`, `reboot_required`). Free-form execution stays confined to the existing live-command feature (admin-only, allowlisted, audited).

S2-3 Agentless patch operations [NEW]

- Linux: package inventory, update check, apply updates, reboot-required detection via SSH using package-manager templates already implemented in agent managers (reuse logic, not copy).
- Windows: inventory and update orchestration via WinRM/PowerShell (Windows Update APIs, winget for apps) with cleanup of any temporary files.
- Network devices: read-only config retrieval over SSH for audit (patching devices is out of scope here).
- Concurrency limits, per-host locks and backoff to avoid overload.

S2-4 Asset onboarding [EXTEND]

- Methods: single IP, CIDR range discovery, IP list import, FQDN/domain, Active Directory sync (LDAP code exists), cloud inventory connectors (optional).
- Mode selection per asset: `agent`, `agentless`, `both` (e.g. agentless scan, agent patch).
- Asset attributes: criticality, group, site, owner, tags, environment, "internet exposed" flag.
- Duplicate detection (existing `(ip, hostname)` constraint) extended to match across transports.

S2-5 Relay Service [NEW]

- A small service installable on any supported OS; registers with the master using a one-time enrolment token, then mutual TLS.
- **Jump mode:** relays connector traffic to assets in an isolated segment.
- **Distribution mode:** also caches patch content locally with checksum verification; assets fetch from the relay.
- Resilience: relay keeps working through short master outages for already-scheduled jobs; resumes content downloads; reports health and cache status.
- Capacity controls: bandwidth caps, cache quotas, retention, pre-seeding for offline relays.

S2-6 Network model [NEW]

- Sites and networks entities; assets belong to a site; each site declares its route (direct, via relay X). The connector layer resolves the path automatically.
- Firewall helper page: for each topology, list required ports and directions (the "no inbound rules for agents" story).

S2-7 Frontend [EXTEND]

- Add-asset wizard with transport choice, credential selection and connectivity test; relay management page; per-asset "how we reach this host" panel.

S2-8 Tests [NEW]

- SSH tests against containerized hosts; WinRM tests against recorded transcripts plus at least one real Windows target in a manual verification checklist.
- Relay failure injection: relay down mid-job, cache corruption, slow link.

Acceptance gate (Stage 2 → 34%)

1. The same patch job runs successfully against one asset via each transport (agent, SSH, WinRM, relay) with identical results and evidence.
2. A host in an isolated segment is scanned and patched through a relay without any inbound rule to the master.
3. Credentials are never present in logs, API responses, evidence bundles or error messages (automated test searches for planted canary secrets).
4. A relay's cache serves N assets while the master sees one download per patch.
5. Killing the relay mid-job leaves the job in a defined, recoverable state.

Evidence required

Transport matrix test report, canary-secret leak test, relay bandwidth measurement, failure-injection results.

---

8. Stage 3 — Vulnerability intelligence (34 → 48)

Goal

Know what is vulnerable, how urgently, and which patch fixes it — without depending on an external scanner.

Why here

Discovery and prioritization (G3, G4) are what connect "patching" to "risk" and feed the Reliability Engine and Proof Loop later.

Principle

Use open, maintained data; own the matching, scoring and workflow. Do not write a proprietary vulnerability knowledge base from scratch.

Work packages

S3-1 Normalized vulnerability model [EXTEND]

- Tables: `vuln_sources`, `vulnerabilities` (CVE id, title, summary, CVSS vectors/scores, CWE, published/modified, references), `vuln_affected` (ecosystem, product/package, version ranges, fixed versions, source), `exploit_signals` (exploit likelihood score, known-exploited flag, source, as-of date), `vuln_patch_map` (vulnerability ↔ patch/KB/package version).
- Keep the existing `cves` table working through a compatibility view until migration completes.

S3-2 Feed ingestion [EXTEND]

- Sources (each behind a plugin interface): NVD, OSV, distro advisories (Debian, Ubuntu, RHEL-family, SUSE, Alpine, Arch), Microsoft security updates (KB ↔ CVE), CISA Known Exploited Vulnerabilities, an exploit-probability dataset (EPSS-class), vendor advisories for top applications.
- Ingestion is **incremental, idempotent and signed-bundle compatible** (Stage 7 will feed the same importer from offline bundles).
- Health page: last sync per source, record counts, errors, staleness warnings.
- **[VERIFY]** each source's terms and rate limits before enabling by default.

S3-3 Inventory-based detection [NEW]

- Collect: OS name/build, kernel, installed packages and versions (per package manager), Windows installed updates and applications, application versions, running services.
- Matching engine per ecosystem: version comparison semantics must be correct per scheme (dpkg, rpm, apk, pacman, Windows build/KB, semver, generic). Build a table-driven test set of tricky version comparisons.
- Output: `host_vulnerabilities` (host, vulnerability, affected component, installed version, fixed version, first seen, last seen, status: open / mitigated / fixed / accepted / false-positive).
- Backport awareness: distro advisories must beat naive upstream version matching to avoid false positives.

S3-4 Unauthenticated scanning [NEW]

- Async TCP connect scan with rate limits and scope enforcement, service/banner identification, TLS inspection (protocol versions, cipher weaknesses, certificate expiry and validity, hostname mismatch), HTTP security header checks, common exposure checks (e.g. SMB signing, anonymous access).
- **Safety rules:** scans only run against targets inside declared scope; each scan requires the "authorised assets" attestation; default non-intrusive; rate limiting and exclusion lists are mandatory features, not options.
- Output feeds the same `host_vulnerabilities` table with source `network_scan`.

S3-5 Risk Engine (PMRS) [NEW]

- PatchMaster Risk Score combining: base severity, exploit likelihood, known-exploited flag, exposure (internet-facing, reachable services), asset criticality, compensating controls, patch availability.
- Weights are **tunable per tenant** and every score shows its breakdown (see Appendix B).
- Backtest harness: replay historical vulnerabilities against the formula and compare ordering with known-exploited sets, so tuning is evidence-based.

S3-6 SLA and remediation workflow [EXTEND]

- SLA policy by PMRS band and asset criticality (reuse `sla.py`); violation detection, owner assignment, accepted-risk with expiry and approver, false-positive handling (reuse `remediation.py`).
- Link each open vulnerability to the recommended action: patch, mitigation, config change or accept.

S3-7 Mitigation playbooks [NEW]

- For vulnerabilities with no patch, or a patch flagged risky by the Reliability Engine, offer vetted mitigations: disable service, firewall rule, config setting, registry change, feature removal.
- Each playbook is versioned, reversible and tested; applying one produces evidence like a patch.

S3-8 UI [EXTEND]

- Replace the CVE page with a risk-first view: PMRS-sorted list, filters (asset group, exposure, exploited, fix available), per-vulnerability detail (evidence, affected assets, remediation options), exports, saved views.

S3-9 Tests [NEW]

- Version-comparison golden tests; backport false-positive corpus; feed-ingest idempotency; scan scope enforcement; PMRS golden cases and backtest report.

Acceptance gate (Stage 3 → 48%)

1. On the fake fleet, detection output matches a hand-verified expected list with **zero known false positives** in the backport corpus.
2. Unauthenticated scan refuses out-of-scope targets and honours rate limits (tests prove both).
3. PMRS ordering ranks known-exploited issues above higher-CVSS non-exploited ones in the backtest; the breakdown is visible for every score.
4. Every open vulnerability shows a remediation option (patch, mitigation, config or accept) or an explicit "none available" state.
5. Feeds can be re-imported repeatedly with no duplicates and no data loss.

Evidence required

Detection diff report, scan-scope test results, backtest report, ingestion idempotency run.

---

9. Stage 4 — Patch engine v3 (48 → 60)

Goal

One patch model for every platform and application type, with real endpoint-friendly reboot handling, wide third-party coverage and macOS.

Why here

Patching must be solid *before* the Reliability Engine and Topology Orchestrator depend on its data and behaviour.

Work packages

S4-1 Unified patch catalog [EXTEND]

- Single `patch_catalog` abstraction across OS patches, application updates, drivers/firmware (where supported) and scripted updates.
- Fields: vendor, product, identifier (KB/package/version), severity, type (security/bugfix/definition/feature), size, reboot requirement, supersedes/superseded-by, dependencies/conflicts, release date, fixed vulnerabilities, source and checksum.
- Reuse `dependency_resolver.py`; promote it from a standalone library to the catalog's resolution step with tests for cycles and conflicts.

S4-2 Deployment phases [EXTEND]

- Standard phases for every job on every transport: `initiation (prerequisites)` → `download` → `integrity verification` → `apply` → `validation`, each with its own status, remark and error code.
- Failure remarks must name the phase and reason so operators can fix and retry.
- Integrity: verify checksums/signatures against the catalog; refuse tampered payloads; record results as evidence.

S4-3 Policy engine upgrade [EXTEND]

- Policy = target (rules, groups, exclusions) + patch filter (severity, vendor, package, type, PMRS band) + reboot behaviour + approval mode + schedule + retry window + ring/gate set.
- Modes: *deploy all except declined* and *deploy only approved*.
- Retry window: devices that were offline are patched when they return, within the policy's window.
- Evaluate policies continuously, not only at schedule time, when new content arrives.

S4-4 Reboot and user experience [EXTEND]

- Reboot modes: none, scheduled (outside business hours), graceful with custom message.
- Graceful mode: user-visible notification, deferral options, maximum deferral count, forced reboot afterwards. Agent shows prompts; agentless hosts use scheduled mode only.
- Servers: never prompt interactively; use maintenance windows and topology rules.

S4-5 Third-party application coverage [NEW]

- Catalog of the most common applications (browsers, PDF readers, collaboration tools, runtimes, compression and media tools, remote-access tools). Build an *update definition* format (detection rule, version source, installer URL/pattern, silent switches, checksum source, uninstall/rollback notes).
- Start with a small, high-quality set; grow by contribution. Quality bar: every definition has a test on a fixture and a documented verification method.
- Windows: winget/MSI/EXE with silent installs; Linux: native repos; macOS: package/installer where licensing allows.
- **[VERIFY]** redistribution and download terms for each vendor; prefer pulling from vendor sources into the customer's cache instead of redistributing binaries.

S4-6 macOS support [NEW]

- Agent for macOS (Intel and Apple silicon), software inventory, system update checks, app updates, reboot handling, MDM-friendly installation notes.
- **[VERIFY]** platform constraints (signing/notarization, privacy permissions, restrictions on system update automation).

S4-7 Rollback unification [EXTEND]

- A single `rollback` action per job that selects the right mechanism (snapshot restore, package downgrade, uninstall, script) from the restore-point capabilities defined in Stage 1.
- Rollback is itself a gated, evidenced job.

S4-8 Scheduling and windows [EXTEND]

- Merge `schedules`, `maintenance` and blackout logic into one scheduling service with conflict detection, timezone correctness and previews. Use `timezone_utils.py` everywhere.

S4-9 Legacy Unix parity [EXTEND]

- Bring AIX, HP-UX and Solaris managers to the same phase/evidence model and test them with recorded transcripts; document supported levels and known limits honestly.

Acceptance gate (Stage 4 → 60%)

1. The same policy patches Windows, Debian-family, RHEL-family and macOS fixtures through the unified catalog and phases.
2. A tampered payload is refused at the integrity phase and the evidence bundle shows why.
3. A graceful reboot shows the custom message, honours deferral limits, then reboots.

4. An offline host is patched after it returns, inside the retry window; outside the window it is flagged, not silently skipped.
5. At least the agreed starter set of third-party apps updates silently on a clean machine with verified versions.

Evidence required

Per-platform job logs and bundles, integrity-failure case, reboot UX recording, third-party app verification matrix.

---

10. Stage 5 — Configuration audit and compliance (60 → 70)

Goal

Measure configuration against benchmarks, explain every failure, and fix or track it.

Why here

Configuration audit (G5) shares the connector, inventory and evidence plumbing built earlier and feeds drift detection and compliance reports.

Principle

Reuse maintained benchmark content and engines; own the workflow, tailoring, evidence and remediation.

Work packages

S5-1 Audit content model [NEW]

- Entities: `audit_profiles` (name, platform, version, source, license note), `audit_controls` (id, title, description, rationale, severity, check definition, remediation, references, framework mappings), `audit_runs`, `audit_results` (host, control, status: pass/fail/not-applicable/error, observed value, expected value, evidence ref).
- Import pipeline for open content (SCAP/OVAL and STIG-class sources) into this model. **[VERIFY]** the licence of every content source; for commercially licensed benchmark content, support *customer-supplied* import instead of bundling.

S5-2 Check engine [NEW]

- Check types: file content/permissions, registry value, service state, package installed/absent, kernel parameter, sysctl, user/group policy, command output match, configuration-file parameter.
- Execution through the connector layer (agent or agentless); results normalized.
- Optional integration with an open SCAP evaluator for content that requires it; keep a native YAML engine for custom checks (extend the existing YAML policy format).

S5-3 Custom profiles and tailoring [EXTEND]

- Clone a profile; change thresholds (e.g. minimum password length); disable controls with justification and approver; add custom controls; version profiles; diff between versions.

S5-4 Network device audit [NEW]

- Two sources: live read-only retrieval over SSH, or uploaded configuration files.
- Parser per vendor family with a rule format (match blocks, required/forbidden lines, ordering), starting with the most common families; unknown families get a generic regex-based mode.

S5-5 Remediation [EXTEND]

- Each failing control shows: why it failed (evidence), impact, how to fix, and where possible an automated remediation (script/policy) that is reversible, gated and evidenced.
- Bulk remediation by control across a group, with canary.

S5-6 Compliance reporting [EXTEND]

- Map controls to frameworks (e.g. ISO 27001, PCI DSS, NIST SP 800-53, CERT-In guidance and RBI-style cyber-security expectations where relevant). **[VERIFY]** current versions of each framework and the exact control numbering before publishing mappings.
- Posture by framework, group, site; trend; exceptions list with expiry; export (PDF, CSV, JSON). Reuse `compliance.py` and `reports.py`.

S5-7 Drift integration [EXTEND]

- Promote `drift_detector.py` baselines to be derivable from an audit profile; drift events create findings and can trigger Proof Loop holds.

S5-8 Tests [NEW]

- Golden control tests with compliant and non-compliant fixtures per platform; parser corpora for network configs; tailoring diff tests.

Acceptance gate (Stage 5 → 70%)

1. A benchmark profile runs against Windows and Linux fixtures with results matching a hand-verified expected set.
2. A custom threshold change alters the result and is recorded in the profile version history.
3. A network config file audit produces correct pass/fail for a published test corpus.

4. Auto-remediation of a control is reversible and appears in the evidence bundle.
5. Compliance report totals reconcile with underlying control results (automated reconciliation test).

Evidence required

Expected-vs-actual result diff, parser corpus report, remediation round-trip proof, report reconciliation output.

---

11. Stage 6 — Topology Orchestrator and Offline Reliability Engine (70 → 82)

Goal

Complete differentiators D3 and D4: patch in the right *order* and with the right *confidence*, fully offline.

Why here

These need the history, vulnerability context and uniform patch model produced by earlier stages.

Part A — Topology Orchestrator (D4)

S6-1 Service topology model [NEW]

- Entities: `services` (name, type, criticality), `service_members` (host, role: primary/replica/node/worker/load-balancer/member, weight), `service_dependencies` (service → service), `health_checks` (type, target, expected result), `drain_actions`.
- Sources: manual definition, import from tags/groups, optional discovery hints (listening ports, cluster configuration).

S6-2 Orchestration plans [NEW]

- A plan is an ordered, gated graph of steps generated from topology + patch set + policy: `pre-checks` → `drain` → `patch` → `reboot` → `verify` → `rejoin` → `post-checks`.
- Rules: never take down more members than the service's tolerance; respect dependency order; stop on first failed gate; resume or roll back safely.
- Plans are previewable (dry-run) and approvable before execution; all steps become evidence.

S6-3 Adapters v1 [NEW]

- **PostgreSQL HA:** replicas first, check replication lag, controlled switchover, patch former primary, rejoin and verify. Support the common HA managers/proxies used in the field (e.g. Pgpool-II and Patroni-class setups) through adapter interfaces. **[VERIFY]** supported versions and commands per manager.

- **Kubernetes:** cordon, drain with disruption-budget awareness, patch node, uncordon, verify readiness; one node (or configured batch) at a time.
- **Load-balanced pools (nginx/HAProxy-class):** remove from pool, drain connections, patch, health check, re-add.
- **Generic:** pre/post hooks plus health command for anything else, reusing `rolling_restart.py` wave logic and `hooks`.

S6-4 Safety rails [NEW]

- Quorum and minimum-healthy-member guards; "last member" protection; automatic pause on elevated error rate; manual "stop now" that leaves the cluster in a known state.
- Failure injection tests are mandatory (primary dies mid-switchover, replica lag spikes, drain times out).

Part B — Offline Reliability Engine (D3)

S6-5 Outcome dataset [EXTEND]

- Capture every patch outcome: host features, patch features, result (success, failure, rollback, post-patch health breach), phase of failure, duration, drift observed. Source: job history, host timeline, Proof Loop gates, evidence.
- Privacy and tenancy: data never leaves the master; per-tenant models unless the operator opts into pooled training.

S6-6 Rule-based scorer v1 [NEW]

- Start explainable and useful with little data: heuristic features such as patch type (kernel, driver, firmware, database/runtime), required reboot, known-problem flags from vendor notices, host role criticality, prior failures on the same OS build or hardware, recency of release, conflicts with installed software, resource headroom.
- Output: a 0–100 **reliability score** plus the top contributing factors and a recommended strategy (deploy now / canary only / extra testing / defer).

S6-7 Learned model v2 [NEW]

- Per-tenant statistical model trained on the outcome dataset. Start with simple, well-calibrated approaches (smoothed failure rates per patch × OS family × role, then gradient-boosted trees when data allows). Calibration and confidence intervals are mandatory; a low-confidence score says so.
- Cold start: blend rule-based and learned scores by sample size.
- Offline training and scoring inside the master; model files are versioned and signed.
- **Guardrail:** the score informs gates and recommendations; it never silently overrides a required gate.

S6-8 Integration with the Proof Loop [EXTEND]

- Reliability score sets the *default* ring sizes and gate strictness (e.g. low reliability → smaller canary, longer observation, restore drill required).
- Operators can override; overrides are evidence.

S6-9 Feedback and monitoring [NEW]

- Track model calibration (predicted vs actual failure rates), drift in feature distributions, and disagreement with operator decisions. Show these on a model-health page.

Acceptance gate (Stage 6 → 82%)

1. A PostgreSQL HA fixture is patched with zero client-visible downtime beyond the defined switchover, with replication-lag gates enforced.
2. A Kubernetes fixture is patched node by node honouring disruption budgets; an injected drain timeout stops the plan safely.
3. Failure-injection tests for every adapter pass and leave the service in a known, documented state.
4. The reliability score is explainable for every patch and its calibration report meets the agreed threshold on replayed history. **[VERIFY]** the threshold with real data before fixing a number.
5. With the network cable pulled, scoring, training and gating all still work.

Evidence required

Adapter failure-injection logs, downtime measurement, calibration report, offline-operation test.

---

12. Stage 7 — Trusted Content Chain, scale and resilience (82 → 92)

Goal

Complete D5 and make the platform fast, highly available and recoverable.

Part A — Trusted Content Chain (D5)

S7-1 Content Builder (online side) [NEW]

- Runs where internet exists. Fetches vulnerability feeds, vendor advisories, patch metadata and (where licensing permits) patch binaries; normalizes; builds **signed bundles**.
- Bundle = manifest (contents, versions, source URLs, fetch timestamps, hashes) + payloads + signature(s). Support *full* and *delta* bundles.

S7-2 Signing and key management [NEW]

- Ed25519 signing keys with a documented hierarchy (root key offline, intermediate signing keys, rotation and revocation lists).
- Key ceremony document; public keys embedded in the product; revocation lists are themselves signed bundles.

S7-3 Import Verifier (offline side) [NEW]

- Verifies signature, chain to trusted root, manifest hashes, version monotonicity (no silent downgrade), and freshness policy before anything is accepted.
- Quarantine area for failed bundles with a clear diagnostic; nothing in quarantine is ever served.
- One-way by design: the import path never needs an outbound connection.

S7-4 Provenance and SBOM [NEW]

- Every patch binary and feed record carries provenance: origin URL, retrieval time, hash, signer. Export provenance as part of evidence bundles. Generate an SBOM for the PatchMaster product itself.

S7-5 Zones [NEW]

- Multiple isolated enclaves, each with its own content stream, trust roots and schedules, administered consistently. A bundle may be targeted to specific zones.

S7-6 Linux local repositories [EXTEND]

- First-class "internal repository" support for restricted Linux estates: mirror management (`mirror_repos.py`), retention, snapshotted repository states per patch cycle so a job always installs from a fixed, evidenced repo state.

Part B — Scale and resilience

S7-7 Performance engineering [EXTEND]

- Define target fleet sizes and job rates; build a load generator using the seed-data generator; profile and fix hot paths (heartbeat intake, host list, CVE queries, dashboard). Use the existing performance-index work as the starting point.
- Backpressure: queue limits, per-tenant fairness, rate limits on heavy endpoints.

S7-8 High availability [EXTEND]

- Stateless backend nodes behind a load balancer (HA compose exists); database replication and automated failover; shared artifact storage; leader election for schedulers so only one node runs each background task.
- Test: kill any single component without losing jobs or corrupting evidence.

S7-9 Platform backup and disaster recovery [NEW]

- Backup/restore for the master itself (database, vault keys, signing keys, content store, configuration) with a restore *drill for the platform*, applying the product's own philosophy to itself.

S7-10 Observability [EXTEND]

- Metrics, structured logs and traces for every stage of the Proof Loop; SLOs for API latency, job start latency, heartbeat freshness; alert rules for stuck jobs, stale content, expiring keys and certificates.

S7-11 Data lifecycle [EXTEND]

- Retention policies for jobs, results, logs and evidence (with legal-hold support), archival, and integrity-preserving pruning.

Acceptance gate (Stage 7 → 92%)

1. A bundle built online, carried across, verified and imported offline updates feeds and patch metadata with full provenance; each of these tamper cases is rejected: altered payload, bad signature, revoked key, replayed older bundle, expired freshness.
2. A job installs from a snapshotted repository state and the evidence shows the exact state used.
3. Load test meets the agreed targets with documented headroom. **[VERIFY]** targets with the intended customer profile.
4. Killing any single node (backend, database primary, relay) loses no job and corrupts no evidence.
5. A full platform restore on a clean machine passes the platform restore drill and the evidence chain still verifies.

Evidence required

Tamper-case matrix results, load-test report, chaos-test report, platform DR drill output.

---

13. Stage 8 — Productization and release (92 → 100)

Goal

A product a customer can buy, install, upgrade, operate, support and audit.

Work packages

S8-1 Installers and upgrades [EXTEND]

- Container and bare-metal installers for the master, relay and agents on every supported platform; preflight checks; automatic backup before upgrade; automatic rollback on

failed upgrade; documented downgrade path.

- Signed release artifacts with checksums; `pm-evidence verify` and the content verifier ship as standalone tools.

S8-2 Licensing and editions [EXTEND]

- Feature flags for each differentiator and module; hardware binding with an offline activation path for air-gapped sites; clear grace and expiry behaviour that never blocks *rollback or evidence export*.

S8-3 Multi-tenancy completion [EXTEND]

- Row-level isolation verified by automated cross-tenant tests across every router and background job; tenant-scoped content, credentials, models and evidence keys.

S8-4 Identity and access [EXTEND]

- LDAP/OIDC finalization, MFA, fine-grained roles (separation of duties: requester ≠ approver), session controls, break-glass accounts with heavy audit.

S8-5 Reporting and dashboards [EXTEND]

- Executive, operations and auditor views: risk trend, SLA compliance, patch success and rollback rates, evidence coverage, compliance posture, topology health. Scheduled report delivery (email/webhook).

S8-6 Integrations [EXTEND]

- Harden Jira, ServiceNow, Slack, Splunk, Sumo Logic and webhook plugins; add SIEM export of evidence events; public API documentation (OpenAPI) and a stable versioned API contract; GraphQL read model kept consistent.

S8-7 Documentation and training [NEW]

- Admin guide, operator guide, security guide, architecture guide, runbooks, troubleshooting trees, auditor guide ("how to verify an evidence bundle"), API reference, release notes, and a training path.

S8-8 Certification readiness [NEW]

- Map controls for the product's own security posture (secure development, vulnerability handling, access control, logging) to the standards customers will ask about; collect the evidence continuously rather than at the end. **[VERIFY]** which certifications the target customers actually require.

S8-9 Pilot hardening [NEW]

- Run with pilot customers' real environments (read-only first), collect defect and usability data, fix, and re-run the Reference Scenario (Section 16) in their environment.

S8-10 Release process [NEW]

- Release checklist, version policy, deprecation policy, supported-platform matrix, vulnerability disclosure process, support runbooks and escalation paths.

Acceptance gate (Stage 8 → 100%)

1. The Reference Scenario passes end to end on a clean install, a customer-like environment and an air-gapped environment.
2. Every acceptance gate from Stages 0–7 is still green on the release candidate (regression run).
3. An external reviewer (not the authors) can install, run a patch job and verify an evidence bundle using only the documentation.
4. Upgrade from the previous release and rollback of a failed upgrade both succeed on the test matrix.
5. Open defects are triaged; zero open critical/high security findings.

Evidence required

Release candidate test report, reviewer's installation log, upgrade/rollback matrix, security scan results, signed release artifacts.

---

14. Cross-cutting tracks (run alongside every stage)

14.1 Security

- Threat model kept current; every new endpoint, connector and parser gets a review entry.
- Agent trust: per-host tokens with rotation and revocation; optional mutual TLS; signed agent updates only; the agent refuses unsigned or downgraded builds.
- Input handling: strict validation at every boundary, parameterized queries, output encoding, file-upload limits and content sniffing, SSRF protection for any URL the server fetches, safe archive extraction (no path traversal).
- Secrets: vault only; no secrets in logs, URLs, evidence or error text (canary-secret tests in CI).
- Least privilege: agents and relays run as non-root where the platform allows; privileged operations go through documented elevation paths.
- Supply chain: pinned dependencies, SBOM, signed releases, reproducible builds as far as practical.
- Abuse resistance: rate limits, lockouts, anomaly alerts on admin actions.

14.2 Quality engineering

- Test pyramid: unit → API → integration (fake fleet) → E2E (UI) → chaos.
- Every bug fix lands with a regression test that fails before the fix.
- Golden-file tests for parsers, version comparison, scoring and reports.
- Contract tests between master and agent/relay so versions can differ safely within the supported skew.
- Mutation or equivalent test-strength checks on the most critical modules (gates, signature verification, scoring).

14.3 Observability

- Correlation IDs from UI click to agent action to evidence item.
- Dashboards for fleet, jobs, gates, content freshness, key/certificate expiry and model health; alerts that name an owner and a runbook.

14.4 UX and accessibility

- Task-based navigation (Find risk → Plan → Prove → Report) instead of a flat module list.
- Every destructive action shows a preview/dry-run and its undo path.
- Accessibility and keyboard support, dark/light themes (tokens already exist), and clear empty/error states.
- Evidence and verification screens must be understandable by an auditor with no product training.

14.5 Documentation

- Docs are part of "done." A work package without updated docs, runbook entries and examples is not done.
- Single source for each topic; generated API reference; changelog entries per work package.

14.6 Compliance and legal hygiene

- Track licences of every dependency and content source; maintain a third-party notices file.
- Document scanning authorisation requirements and the customer's responsibility for scope.
- Data classification: what the product stores (inventory, credentials, evidence) and how long.

---

15. Rules for the coding agent (Mimo)

These rules apply to **every instruction, prompt and task given to the coding agent** while executing this plan.

15.1 Non-negotiable rules

1. **No vibe coding allowed.** Every change must be driven by a stated requirement from this plan, a failing test, or a reproduced defect. No speculative rewrites, no "this should work" edits, no guessing at behaviour that can be checked.
2. **Use the Ponytail and REA plugins wherever they apply, and paste their output as evidence** in the work package's evidence section and in the change description. A claim without pasted plugin output (where a plugin applies) is treated as not done.
3. **Read before writing.** Open the files being changed and their callers; summarize what you found before editing.
4. **Tests first for behaviour.** For new behaviour, write the failing test first. For refactors, write characterization tests first.
5. **One work package per change set.** Keep changes small, reviewable and reversible.
6. **No unrelated edits.** Do not reformat, rename or "tidy" outside the work package.
7. **No secrets, ever,** in code, tests, fixtures, logs or evidence. Use the vault and test-only generated values.
8. **Stop and ask** when a requirement is ambiguous, a **[VERIFY]** item cannot be resolved from the repo, or a change would touch security boundaries (auth, license, signing, vault) beyond the work package's scope.
9. **Honest reporting.** State what was run, what passed, what failed and what was not run. Never mark a gate passed without its evidence.

15.2 Definition of done (every work package)

- [ ] Requirement and acceptance criteria quoted from this plan.
- [ ] Failing test(s) written first and shown failing, then passing.
- [ ] Full relevant test suites green; CI gates green.
- [ ] Security considerations written (inputs, privileges, secrets, failure modes).
- [ ] Migration included and tested where the schema changed.
- [ ] Docs, runbook and changelog updated.
- [ ] Ponytail/REA output pasted where applicable.
- [ ] Rollback path described (how to undo this change).
- [ ] Evidence section filled in with real command output, not paraphrase.

15.3 Standard task template for the agent

```text
TASK: <work package id and title from plan.md>
CONTEXT: <files and modules involved; link to relevant plan sections>
REQUIREMENTS: <copied from the work package>
ACCEPTANCE CRITERIA: <copied from the stage gate, narrowed to this package>
RULES: No vibe coding allowed. Use Ponytail and REA plugins where they apply and paste
       their output as evidence. Tests first. Smallest possible change. No unrelated e
DELIVERABLES: code, tests, migration (if any), docs, evidence section, rollback note.
STOP CONDITIONS: ambiguity, unresolved [VERIFY], or scope creep into security boundari
```

15.4 Review checklist (human reviewer)

- Does the change map to exactly one work package?
- Do tests prove the acceptance criteria rather than the implementation?
- Are failure paths tested (timeouts, partial failure, bad input, cancel)?
- Could this leak a secret or widen an attack surface?
- Is the undo path real and tested?
- Is the evidence real output?

---

16. Reference scenario (the end-to-end acceptance test)

This single scenario is run (and extended) at the end of every stage as far as that stage allows, and in full at Stage 8. It is also the demo that proves the product is different.

16.1 The estate

Zone	Contents	Reach
HQ	PostgreSQL HA pair (primary + replica behind a proxy), a Kubernetes cluster (several worker nodes), a load-balanced web pool, Windows servers, Windows and macOS endpoints, Debian/RHEL servers	Agent for endpoints; SSH/WinRM for servers
Branch	Mixed Linux and Windows endpoints on a slow link	Relay in distribution mode
Enclave	RHEL and AIX hosts with no internet	Signed offline bundles; relay in jump mode

16.2 The story

1. **Content arrives.** A signed bundle containing a newly published vulnerability flagged as actively exploited is built online, carried to the enclave and imported. The verifier accepts it and shows provenance.
2. **Detection.** Within the same run the estate is matched; affected hosts appear with PMRS ranking and an exploitation flag.
3. **Prioritization.** The exploited, internet-facing database-adjacent issue ranks above higher-CVSS but unexploited findings; the score breakdown is visible.
4. **Reliability.** The Reliability Engine scores the fix patch as risky for database hosts (kernel and reboot involved) and recommends canary-only plus a restore drill.
5. **Plan.** The Topology Orchestrator produces a preview: replica first, switchover, then former primary; Kubernetes node-by-node; pool members drained one at a time.
6. **Approval.** A requester submits; a different approver approves (separation of duties). Both are evidence items.
7. **Canary.** Canary hosts are patched. **Failure variant A:** an injected health-check failure triggers automatic rollback and the bundle records it. **Pass variant B:** the canary is healthy, drift is clean, the restore drill passes.
8. **Promotion.** Rings promote automatically through gates; the branch pulls content from the relay (the master sees one download); the enclave installs from a snapshotted local repository state.
9. **Evidence.** A signed bundle is produced for the job.
10. **Audit.** An auditor with no network access verifies the bundle: PASS.
11. **Tamper.** One byte is changed: verification FAILS and names the failing item.

16.3 Pass criteria

- Every step above is demonstrated with real output and no manual database edits.
- The failure variant leaves the estate in a known, healthy, documented state.
- Re-running the scenario with a relay outage, a rotated credential and a bad bundle produces safe, explained outcomes.

---

17. Metrics, risk register and decisions

17.1 Product metrics

Metric	Why it matters
Patch success rate; rollback rate	Core quality signals
Restore-proof success rate	Proves the undo path is real
Evidence coverage (% of jobs with a verified bundle)	The thesis, measured
Gate precision (blocked promotions that would truly have failed)	Are gates useful or just noisy?
Operator override rate and reasons	Where trust breaks down
Vulnerability false-positive rate (against curated corpora)	Credibility of detection
PMRS ranking quality (backtest)	Credibility of prioritization
SLA compliance by band	Customer outcome
Mean time to remediate (critical band)	Customer outcome
Relay cache hit rate and master download reduction	Efficiency story
Reliability calibration error	Model honesty
Platform availability and restore-drill pass rate	Operational trust

17.2 Risk register

ID	Risk	Impact	Mitigation
R1	Scope too wide: parity and thesis both attempted at full depth	Nothing finishes well	Risk-driven order; cut parity depth (app catalog size, device families) before cutting differentiators
R2	Gates are noisy and operators override them	Thesis loses credibility	Measure gate precision; start strict only where data supports it; make overrides visible
R3	Windows agentless security and reliability	Incidents at customers	Template-only execution, vault, least privilege, real-target verification checklist
R4	Version-comparison and backport errors cause false positives/negatives	Credibility loss	Golden corpora, distro-advisory-first matching, continuous regression
R5	Benchmark or vendor content licensing	Legal exposure	Customer-supplied import path; legal review; avoid bundling restricted content
R6	Reliability model has too little data	Bad scores	Rule-based first; confidence reporting; never override required gates
R7	Key management mistakes in signing chain	Catastrophic trust failure	Offline root, documented ceremony, rotation/revocation tests
R8	Refactors in Stage 0 regress behaviour	Lost trust, delays	Characterization tests first; behaviour-preserving rule
R9	Competitors close the gap (offline AI scoring, signed content)	Differentiation erodes	Keep the *closed loop with proof* as the moat; publish verification tooling and auditor guide
R10	Topology adapters differ across versions of databases and orchestrators	Unsafe automation	Version-pinned adapters, dry-run mandatory first, failure-injection suite
R11	Single maintainer knowledge concentration	Fragility	Docs as part of done; runbooks; second reviewer on security boundaries

ID	Risk	Impact	Mitigation
R12	Scanning used against unauthorised targets	Legal/ethical harm	Scope attestation, enforced allowlists, audit trail

17.3 Decisions required (record each as an architecture decision)

ID	Decision	Options
A-1	Migration framework	Alembic-class vs keep scripted SQL with a runner
A-2	Signing key hierarchy and storage	Offline root + intermediates; hardware token vs file
A-3	Benchmark content sources	Open SCAP-class content, STIG-class, customer-supplied commercial content
A-4	PMRS default weights and bands	Start from Appendix B, calibrate by backtest
A-5	Minimum supported OS versions per platform	Driven by customer segment and test capacity
A-6	Relay transport	Outbound-only tunnel from relay vs inbound to relay
A-7	Evidence storage format and retention	Archive format, location, legal hold
A-8	Third-party app definition format and contribution model	Declarative files + CI verification
A-9	Reliability model family	Smoothed rates → boosted trees; calibration method
A-10	Edition and licensing structure	Which differentiators are core vs add-on

17.4 Open questions for the product owner

1. Primary customer segment and the OS/application mix that matters most to them?
2. Target fleet size for the first production customers, and for headroom?
3. Is the master always customer-hosted, or will a managed offering exist?
4. Which compliance frameworks do the first customers actually require?
5. Which stateful platforms matter most for the Topology Orchestrator after PostgreSQL and Kubernetes?

18. Appendices

Appendix A — Data model sketches (new or changed)

```sql
```

```
-- Credential vault
CREATE TABLE vault_credentials (
  id UUID PRIMARY KEY, tenant_id UUID NOT NULL, name TEXT NOT NULL,
  kind TEXT NOT NULL,                -- ssh_password|ssh_key|winrm|snmp|device_enable|
  scope_type TEXT NOT NULL,          -- global|group|site|host
  scope_id UUID, username TEXT,
  encrypted_blob BYTEA NOT NULL, key_id TEXT NOT NULL,
  expires_at TIMESTAMPTZ, rotated_at TIMESTAMPTZ, created_by UUID, created_at TIMESTAM
  UNIQUE (tenant_id, name)
);

-- Gates and results
CREATE TABLE gate_definitions (
  id UUID PRIMARY KEY, policy_id UUID, ring_index INT, phase TEXT,   -- pre|post
  gate_type TEXT NOT NULL, params JSONB, required BOOLEAN DEFAULT TRUE,
  timeout_seconds INT, retries INT
);
CREATE TABLE gate_results (
  id UUID PRIMARY KEY, job_id UUID, ring_index INT, gate_definition_id UUID,
  status TEXT, observed JSONB, started_at TIMESTAMPTZ, finished_at TIMESTAMPTZ, artifa
);

-- Evidence
CREATE TABLE evidence_bundles (
  id UUID PRIMARY KEY, tenant_id UUID, job_id UUID UNIQUE,
  bundle_hash TEXT NOT NULL, previous_hash TEXT, signature BYTEA NOT NULL, signer_key_
  storage_ref TEXT NOT NULL, created_at TIMESTAMPTZ
);
CREATE TABLE evidence_items (
  id UUID PRIMARY KEY, bundle_id UUID REFERENCES evidence_bundles(id),
  seq INT, step TEXT, kind TEXT, payload_ref TEXT, item_hash TEXT NOT NULL
);

-- Vulnerability intelligence
CREATE TABLE host_vulnerabilities (
  id UUID PRIMARY KEY, host_id UUID, vulnerability_id TEXT, component TEXT,
  installed_version TEXT, fixed_version TEXT, source TEXT,      -- inventory|network_s
  status TEXT,                                                  -- open|mitigated|fixe
  pmrs NUMERIC, pmrs_breakdown JSONB, first_seen TIMESTAMPTZ, last_seen TIMESTAMPTZ
);
```

```
-- Topology
CREATE TABLE services (id UUID PRIMARY KEY, tenant_id UUID, name TEXT, type TEXT, crit
CREATE TABLE service_members (service_id UUID, host_id UUID, role TEXT, weight INT, PR
CREATE TABLE service_dependencies (service_id UUID, depends_on UUID, PRIMARY KEY (serv

-- Content chain
CREATE TABLE content_bundles (
  id UUID PRIMARY KEY, zone_id UUID, version TEXT, kind TEXT,     -- full|delta
  manifest_hash TEXT, signer_key_id TEXT, imported_at TIMESTAMPTZ, status TEXT   -- ac
);
```

Appendix B — PatchMaster Risk Score (PMRS) starting formula

All inputs are normalized to 0–1. **Defaults are starting points and must be calibrated by the backtest harness (S3-5).**

text

```
base  =  0.25 * (cvss / 10)
       + 0.25 * exploit_likelihood          # EPSS-class probability, 0-1
       + 0.20 * known_exploited             # 0 or 1
       + 0.10 * exposure                    # internet-facing 1.0 | reachable servic
       + 0.15 * asset_criticality           # low 0.25 | medium 0.5 | high 0.75 | cr

adjusted = base * (0.6 if compensating_control else 1.0)
floor    = 0.85 if (known_exploited and exposure == 1.0) else 0.0
PMRS     = 100 * max(adjusted, floor)

Bands:  Critical >= 85 | High 70-84 | Medium 40-69 | Low < 40
Flags:  patch_available (true/false) → drives "patch" vs "mitigate" recommendation
```

Every PMRS display must show the five inputs, the adjustments and the floor rule that applied.

Appendix C — Evidence bundle manifest (v1)

json

```
{
  "schema": "patchmaster.evidence/1",
  "bundle_id": "…",
  "job_id": "…",
  "tenant": "…",
  "created_at": "…",
  "policy": { "id": "…", "version": 7, "hash": "sha256:…" },
  "approvals": [ { "role": "requester", "user": "…", "at": "…" }, { "role": "approver"
  "scope": { "hosts": 42, "rings": 3, "patches": ["KB…", "pkg-1.2.3"] },
  "steps": [
    { "seq": 1, "step": "preflight", "result": "passed", "items": ["sha256:…"] },
    { "seq": 2, "step": "snapshot",  "result": "passed", "restore_point": "…", "capabi
    { "seq": 3, "step": "canary",    "result": "passed", "hosts": ["…"] },
    { "seq": 4, "step": "gates",     "result": "passed", "gates": [ { "type": "service
    { "seq": 5, "step": "drift",     "result": "clean" },
    { "seq": 6, "step": "restore_drill", "result": "passed", "rto_minutes": 4, "rpo_mi
    { "seq": 7, "step": "promotion", "decision": "promote", "reliability": { "score":
  ],
  "content_state": { "bundle_version": "…", "repo_snapshot": "…" },
  "overrides": [],
  "previous_bundle_hash": "sha256:…",
  "bundle_hash": "sha256:…",
  "signature": { "alg": "ed25519", "key_id": "…", "value": "…" }
}
```


Verification rules: recompute every item hash; recompute the bundle hash; check the chain link; verify the signature against a trusted key; confirm every step required by the governing policy is present.

Appendix D — Reliability score v1 (rule-based) feature starters

Starting penalties (subtract from 100; tune with data; show contributions):

Factor	Example penalty
Kernel / driver / firmware update	15–25
Reboot required on a service host	10
Host role: database / cluster member	10
Previous failure of the same patch on the same OS build	20
Vendor-reported known issue (from content)	20
Released very recently (little field exposure)	10
Low disk or memory headroom	10
Conflicting installed software	15
Large payload on slow link	5

Recommendation mapping: ⩾ 80 deploy per policy • 60–79 canary with standard gates • 40–59 canary + restore drill + extended observation • < 40 defer or mitigate.

Appendix E — Platform support matrix (to confirm and keep current)

Platform	Agent	Agentless	Restore capability	Notes
Windows client/server	Yes	WinRM	Snapshot/backup tooling already present **[VERIFY]**	WSUS-aware
Debian/Ubuntu	Yes	SSH	Depends on filesystem/volume tooling	apt manager exists
RHEL family	Yes	SSH	Depends on filesystem/volume tooling	yum/dnf manager exists
SUSE	Yes	SSH	Depends	zypper manager exists
Alpine / Arch	Yes	SSH	Package-level	apk / pacman managers exist
FreeBSD	Yes	SSH	Filesystem snapshots if available	pkg manager exists
AIX / HP-UX / Solaris	Yes	SSH **[VERIFY]**	Platform tooling **[VERIFY]**	Managers exist; test with recorded transcripts and real systems
macOS	New	SSH (limited) **[VERIFY]**	Limited	Stage 4
Network devices	n/a	SSH / config upload	n/a	Audit only

Appendix F — API surface to add (illustrative)

- `POST /api/vault/credentials`, `POST /api/vault/credentials/{id}/test`
- `POST /api/assets/discover`, `PUT /api/assets/{id}/transport`
- `POST /api/relays/enroll`, `GET /api/relays/{id}/health`
- `GET /api/vulns` (risk-first), `POST /api/vulns/{id}/accept`, `GET /api/vulns/{id}/breakdown`
- `POST /api/scans`, `GET /api/scans/{id}`
- `POST /api/audits/run`, `GET /api/audits/profiles`, `POST /api/audits/profiles/{id}/clone`

- `GET /api/jobs/{id}/gates`, `POST /api/jobs/{id}/promote`, `POST /api/jobs/{id}/override`
- `GET /api/evidence/{job_id}`, `GET /api/evidence/{job_id}/bundle`, `POST /api/evidence/verify`
- `POST /api/topology/services`, `POST /api/topology/plans/preview`, `POST /api/topology/plans/{id}/execute`
- `POST /api/content/import`, `GET /api/content/bundles`, `POST /api/content/quarantine/{id}/release`
- `GET /api/reliability/score?patch=…&hosts=…`, `GET /api/reliability/model-health`

All new endpoints: authenticated, RBAC-gated, tenant-scoped, rate-limited, audited, and covered by contract tests.

Appendix G — Glossary

- **Proof Loop** — gated, evidence-driven promotion through rollout rings.
- **Evidence Bundle** — signed, hash-chained record of a patch job.
- **PMRS** — PatchMaster Risk Score.
- **Relay** — jump-host and/or distribution-server component for remote networks.
- **Zone** — an isolated enclave with its own content stream and trust roots.
- **Restore point** — any verified undo mechanism (snapshot, backup, package downgrade, script).

Appendix H — Sources used for market context (re-verify before external use)

- Qualys: AI-Powered Patch Reliability Scoring — https://blog.qualys.com/product-tech/2026/02/18/new-ai-powered-patch-reliability-scoring-predict-patch-impact-before-you-deploy
- Ivanti Predictive Remediation coverage — https://www.the-sourcecode.com/cybersecurity/ivanti-predictive-remediation-patch-failures
- Adaptiva AirGap for OneSite Patch — https://adaptiva.com/press-release/adaptiva-launches-airgap-for-onesite-patch
- Patch management provider comparison (BigFix and others) — https://www.esecurityplanet.com/products/patch-management-service-providers/
- Vulnerability management comparison (Qualys VMDR, Tenable, Rapid7) — https://cipherssecurity.com/best-vulnerability-management-platform-2026/
- Motadata ServiceOps patch management documentation — https://docs.motadata.com/serviceops-docs/technician-user-guide/patch-management/overview

- SecOps platform — demo transcript and architecture material supplied by the product owner.

End of plan. The plan is complete when Stage 8's gate passes and the Reference Scenario runs clean on a fresh install, in a customer-like environment and in an air-gapped environment.